Graph drawing by stress minimization

Laslo Hauschild, Lennart Peters

March 30, 2026

Introduction

Graphs are a prominent object in modern mathematics, useful for abstracting complex reallife situations.

- Railway networks, data structures, geometric objects
- Model relationships: vertices for entities, edges for connections

Example: Two companies with employees as vertices and daily interactions as edges.

- Compare communication hierarchy and department interactions
- Difficulty: meticulously searching through huge lists of vertices and edges

Goal: Represent finite graphs in $\mathbb{R}^2$ and $\mathbb{R}^3$ esthetically.

- Drawing: vertices as points, edges as straight-line segments
- Quickly understand structure and type of a given graph

Graph embedding

Graph definition:

- set of vertices V, set of edges $E \subseteq \{\{u, v\} \mid u, v \in E\}$ then $G = (V, E)$

Our goal is to draw graphs, for our purposes a drawable graph has a finite set of vertices.

- finite $V \implies$ finite E

A graph embedding p is an injective map $p : V \to A$, which maps every $v \in V$. p induces an embedding of E, therefore p is a lossless embedding of G in A.

- $\exists p \iff |V| \leq |A|$

Let us call $H = p(G)$ and choose $A = \mathbb{R}^n, \quad n = 2, 3$

Modeling stress

We add a drawing function $d : F \to \mathbb{R}, \{a, b\} \mapsto \overline{ab}$, giving us a drawing of H.

$W \subseteq \mathbb{R}^n$ essentially randomly chosen, drawings of H are mostly tangled. We use a stress function S that assigns the graph H a real number, the stress in H. Minima of S are our desired representations of G, if possible the global minimum as ideal.

FR model

developed 1991, by Fruchterman and Reingold, following two main principles [4]:

- drawing neighboring vertices near each other
- vertices should not be drawn too near to each other

The FR model uses the following stress-function:

$$S_{FR}(p(G)) = \frac{1}{6} \sum_{\{u,v\} \in E} \| p(u) - p(v) \|^3 - \sum_{\{u,v\} \subseteq V} \log \| p(u) - p(v) \| \tag{1}$$

Observe:

- the further apart neighboring vertices get mapped the bigger the stress
- small distances between vertices let the logarithm explode
- the logarithm dominates at small distances and hinders the graph from collapsing, or converging into a single point

S_{FR} satisfies the two main principles, while being rather quick when implemented into an algorithm.

ERLL model

The Edge Repulsion LinLog model by Andreas Noack, 2003, places its focus on the graph's edges. For $v \in V$, let $\deg(v)$ denote the number of edges v is an endpoint of [1].

$$\begin{aligned} S_{ERLL}(p(G)) = & \frac{1}{2} \sum_{\{u,v\} \in E} \| p(u) - p(v) \| \\ & - \sum_{\{u,v\} \subseteq V} \deg(u) \deg(v) \log \| p(u) - p(v) \| \end{aligned} \tag{2}$$

Observe:

- edges cause both attraction and repulsion
- G of uniform degrees: FR $\approx$ ERLL scaled with $c = \deg(v)^2$
- G of non-uniform degrees: FR repulsion remains the same, but vertices u, v repel each other proportional to $\deg(u)\deg(v)$ in ERLL.

$\implies$ ERLL spreads out "big nodes" compared to FR, usefull in highly non-uniform graphs

Implementation

Plotting a Graph Embedding

```julia
using Plots
using Graphs

function plot_graph(g::SimpleGraph, p::Matrix{Float64})
    plt = plot(legend = false, aspect_ratio = :equal)
    for e in edges(g)
        plot!(plt, [p[1, src(e)], p[1, dst(e)]],
            [p[2, src(e)], p[2, dst(e)]], color = :black)
    end
    scatter!(plt, p[1, :], p[2, :], markersize=5, color = :blue)
    return plt
end
```

A couple notes:

- Dependencies: Graphs [5] and Plots [6] libraries
- Arguments: A graph g and a position matrix p with vertex coordinates
- Order of plotting matters: edges first, then vertices

Plotting in higher dimensions is easy. Just extend the `plot!` and `scatter!` calls by one more argument for the third dimension:

```
function plot_graph_3d(g::SimpleGraph, p::Matrix{Float64})
    plt = plot(legend = false, aspect_ratio = :equal)
    for e in edges(g)
        plot!(plt, [p[1, src(e)], p[1, dst(e)]], [p[2, src(e
            [p[3, src(e)], p[3, dst(e)]], color = :black)
    end
    scatter!(plt, p[1, :], p[2, :], p[3, :], markersize=5, c
    return plt
end
```

In the initialization of all the vectors and matrices, make sure to update the dimensions to 3.

Since the two stress functions explored in the theoretical part is

structured in a similar way, we can implement them in a single function. We just need to pass the edge and vertex stress functions as arguments. To do this, we use anonymous functions:

```
function stress(
    g::SimpleGraph,
    edge_stress_f::Function,
    vertex_stress_f::Function,
    combine_f::Function)

    n = nv(g)
    edge_stress = 0
    for e in edges(g)
        edge_stress += edge_stress_f(src(e), dst(e))
    end
    vertex_stress = 0
```

```
    for i in 1:n
        for j in (i+1):n
            vertex_stress += vertex_stress_f(i, j)
        end
    end

    return combine_f(edge_stress, vertex_stress)
end
```

Implementing the FR and ERLL stress functions is then straightforward:

```
function fr_stress(g::SimpleGraph, p::Matrix{Float64})
    D = pairwise(Euclidean(), p, dims=2)
    edge_stress_f = (i, j) -> D[i, j]^3
    vertex_stress_f = (i, j) -> log(D[i, j] + 10^(-6))
    combine_f = (x, y) -> x / 6 - y
```

```
    return stress(g, edge_stress_f, vertex_stress_f, combine
end

function erll_stress(g::SimpleGraph, p::Matrix{Float64}, deg
    D = pairwise(Euclidean(), p, dims=2)
    edge_stress_f = (i, j) -> D[i, j]
    vertex_stress_f = (i, j) -> deg[i] * deg[j] * log(D[i, j
    combine_f = (x, y) -> x / 2 - y
    return stress(g, edge_stress_f, vertex_stress_f, combine
end
```

Note that we add a small constant to the distance in the logarithm to prevent it from exploding when two vertices are mapped to the same point.

We use the `pairwise` function from the Distances library [2] to compute the distance matrix D. This is way more efficient than

computing the distances in a nested loop.

Also note that the erll stress function takes an additional argument, which is the degree of each vertex. This is used in the calculation, but since the degree of each vertex does not change, it would be inefficient to calculate it inside the vertex stress function.

The final piece in the puzzle is to actually use the plotting and the stress functions to find good embeddings of a given graph. To do this, we use gradient descent. We use the Zygote library [3] to compute the gradients of the stress functions.

With zygote, we can easily differentiate through our stress functions, which are implemented in a very straightforward way. Zygote does this by using source-to-source automatic differentiation, which means that it transforms our code into a form that can be differentiated, and then applies the chain rule to

compute the gradients. This allows us to compute the gradients of our stress functions without having to manually derive them, which can be quite tedious and error-prone.

The principle of gradient descent is the following:

We initialize the positions of the vertices randomly in a 2D space. We then compute the stress and its gradient at each step, and update the positions by some rule. To check for convergence, we either stop after the maximum number of steps has been reached, or if the stress values do not change meaningfully. It is important that we do not check for equality of the stress values, since they are floating-point numbers and can be very close to each other without being exactly equal. Instead, we check if the relative difference is smaller than a small threshold.

This is the simple version of the gradient descent algorithm. One

can imagine what this algorithm does by imagining a hiker being very tired and as such, trying to find the nearest town. To do that, he always goes in the direction of steepest descent. Crucially, he does not have any momentum.

```
function simple_gradient_descent(g::SimpleGraph, stress_fn::
    max_steps::Int64, filename::String, stress_fn_name::Stri

    n = nv(g) # Number of vertices
    p = rand(2, n) # Initial positions

    stresses = Float64[]

    for k in 1:max_steps
        s, (grad,) = Zygote.withgradient(q -> stress_fn(g, c
        push!(stresses, s)
```

```
if length(stresses) > 1 &&
    abs((stresses[end] - stresses[end-1]) / stresses

    println("converged!")
    println("iteration ", k)
    plot1 = plot_graph(g, p)
    plot2 = plot(stresses, label="stress")
    final = plot(plot1, plot2, layout=2)
    savefig(final, filename * "_simple_" * stress_fr
        * "_a" * string(alpha) * "_i" * string(k) *
    break
end

p -= alpha * grad
```

```
        if k % 30 == 0
            plot1 = plot_graph(g, p)
            plot2 = plot(stresses, label="stress")
            display(plot(plot1, plot2, layout=2))
        end
    end
end
```

A couple notes:

- Let s_l be the last stress value and s_{l-1} be the second to last stress value. To calulate if the algorithm converged, we do $\| \frac{s_l - s_{l-1}}{s_{l-1}} \| < \epsilon$. A more natural way might be $\| s_l - s_{l-1} \| < \epsilon$. However, with this approach, if the stress value is different for different functions, i.e. because the nodes are pushed further apart, non-visible changes would be in an order of magnitude

higher, so to account for that, we should use the relative change instead of the absolute change.

- Again, we use anonymous functions here. In this case, to easily swap the stress function, it is passed as an argument instead of being hard-coded.
- The parameters `filename::String` and `stres_fn_name::String` are for easier management of multiple graph drawings at the same time. Earlier versions of that implementation just ouputted the same file, overwriting previous results.

In this version of gradient descent, we not only go a small step in the direction of the gradient, but we also add momentum to the term.

This can be imagined by a ball rolling, which in addition to going downhill, also has some inertia, which makes it roll faster and faster

as it goes down the hill. This can help to escape local minima and find better solutions.

Friction is modelled by β, which says how much of the previous momentum we keep. Because $\beta = 0.9$, we keep 90% of the previous momentum.

```
function nesterov_gradient_descent(g::SimpleGraph, stress_fr
    beta::Float64, max_steps::Int64, filename::String, stres

    n = nv(g) # Number of vertices
    p = rand(2, n) # Initial positions

    stresses = Float64[]

    momentum = zeros(2, n)
```

```
for k in 0:max_steps
    s, (grad,) = Zygote.withgradient(q -> stress_fn(g, c
    new_momentum = beta * momentum - alpha * grad
    push!(stresses, s)

    if length(stresses) > 50 &&
        abs((stresses[end] - stresses[end-1]) / stresses

        println("converged!")
        println("iteration ", k)
        plot1 = plot_graph(g, p)
        plot2 = plot(stresses, label="stress")
        final = plot(plot1, plot2, layout=2)
        savefig(final, filename * "_nesterov_" * stress_
            * "_a" * string(alpha) * "_i" * string(k) *
```

```
            break
        end

        p += new_momentum
        momentum = new_momentum

        if k % 30 == 0
            plot1 = plot_graph(g, p)
            plot2 = plot(stresses, label="stress")
            display(plot(plot1, plot2, layout=2))
        end
    end
end
```

Results

When trying our models, we had some problems with the suggested learning rates. For the FR stress, the suggested learning rates were fine, but for ERLL, the learning rates were way to small, which made convergence very slow. Therefore, we analyzed which learning rate fits best for the different stress values.

Let's start with the simple gradient descent method, fr stress, $\alpha = 0.001$ and the wheel graph 11.

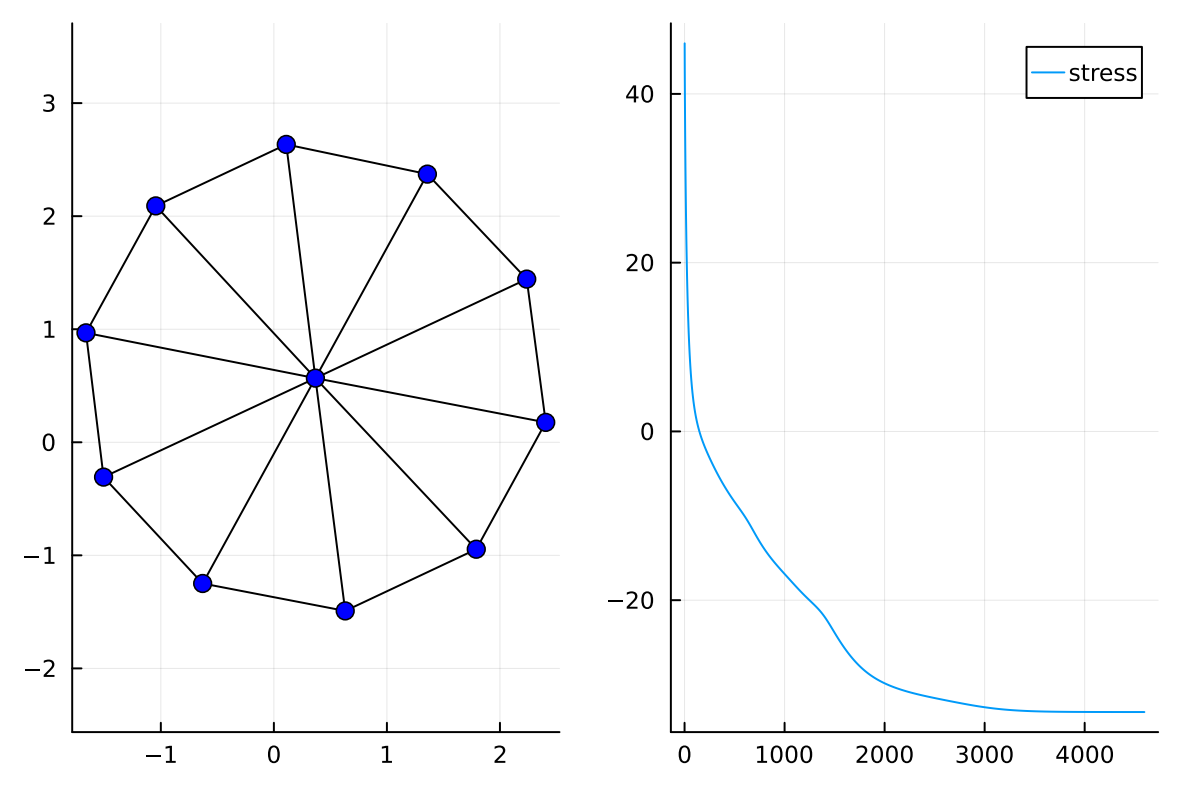

Figure 1: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.001$, Took 4597 Iterations

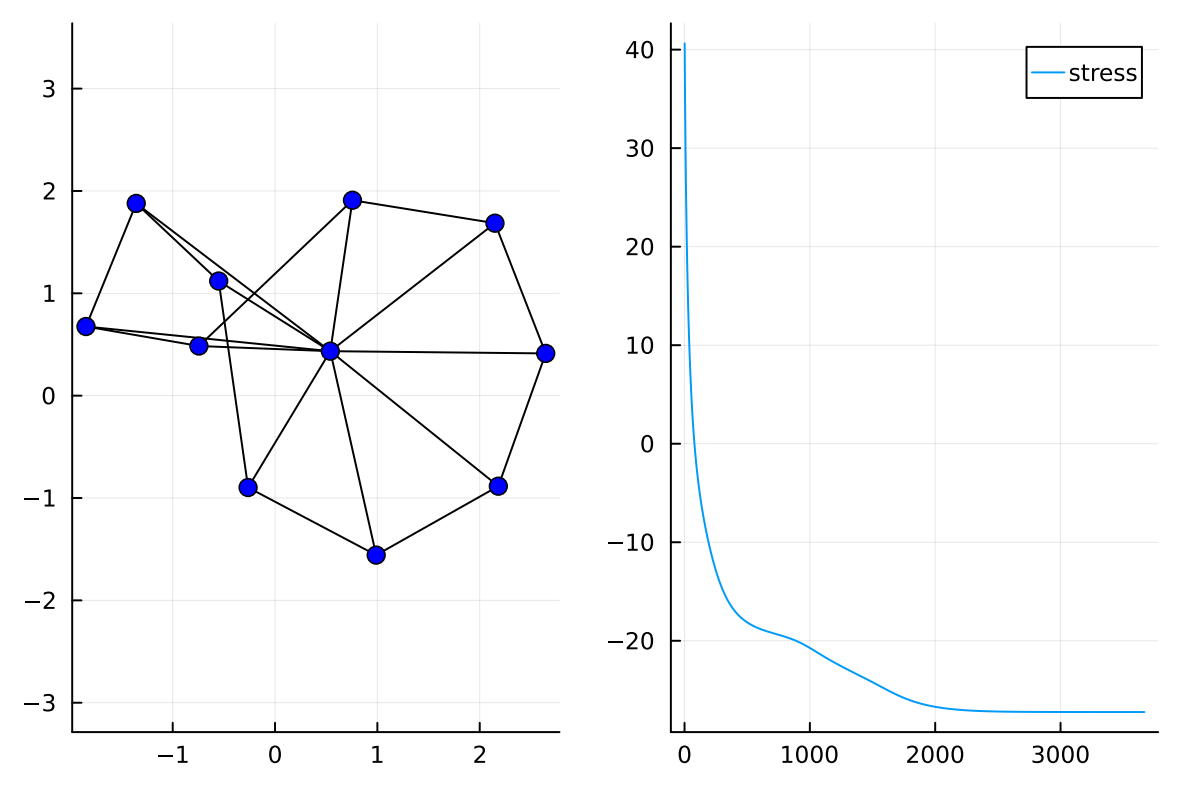

Figure 2: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.001$, Took 3669 Iterations

Almost always, the graph converges to something looking like Figure 1.

In a few cases (less than 10%), we converge into a local minimum like depicted in Figure 2

When we increase the learning rate α, there are interesting results.

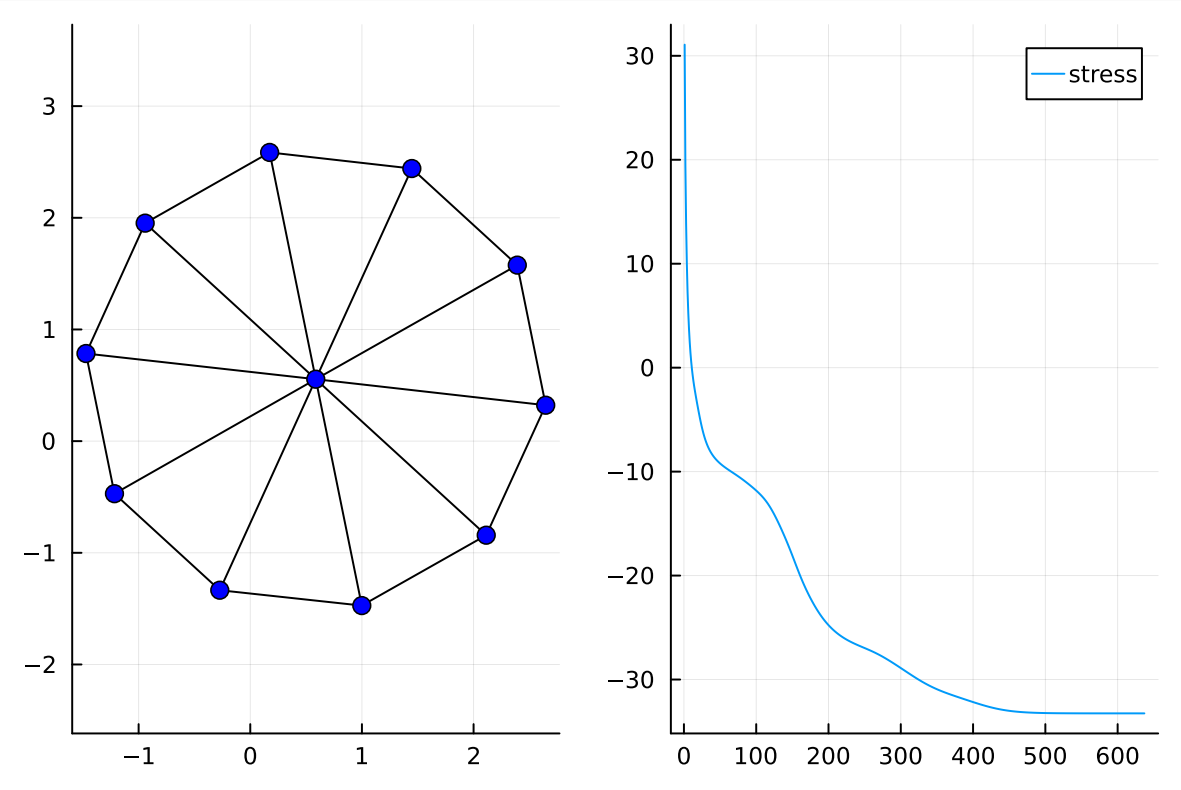

Figure 3: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.01$, Took

428 Iterations

With a learning rate $\alpha = 0.01$, the convergence speed dramatically increases with no visible poorer quality.

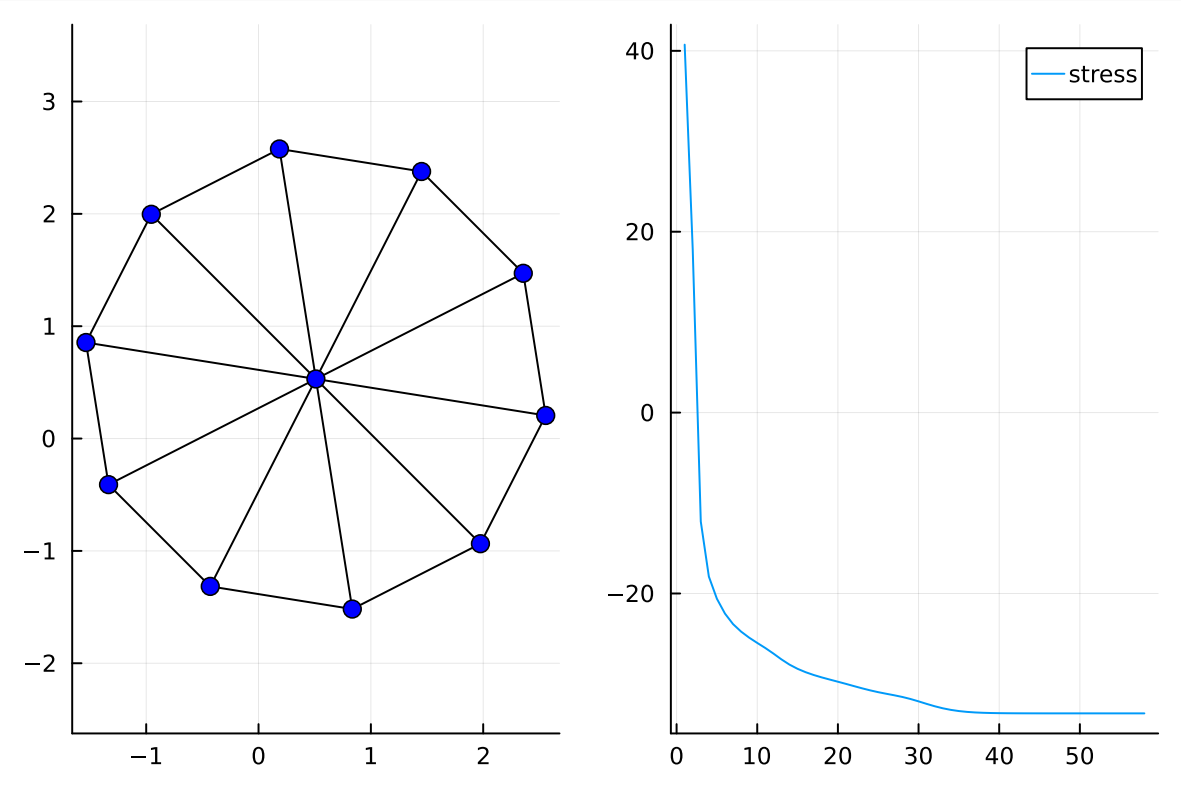

Figure 4: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.1$, Took 57

At $\alpha = 0.1$, the convergence speed increases even more, with quality staying essentially the same. However, the rate of poorly converged graphs increases, however this is difficult to quantify. We also see that stress has actually increased twice, even though we were going "downhill". This is because we are taking too big steps, which makes us overshoot the "valley".

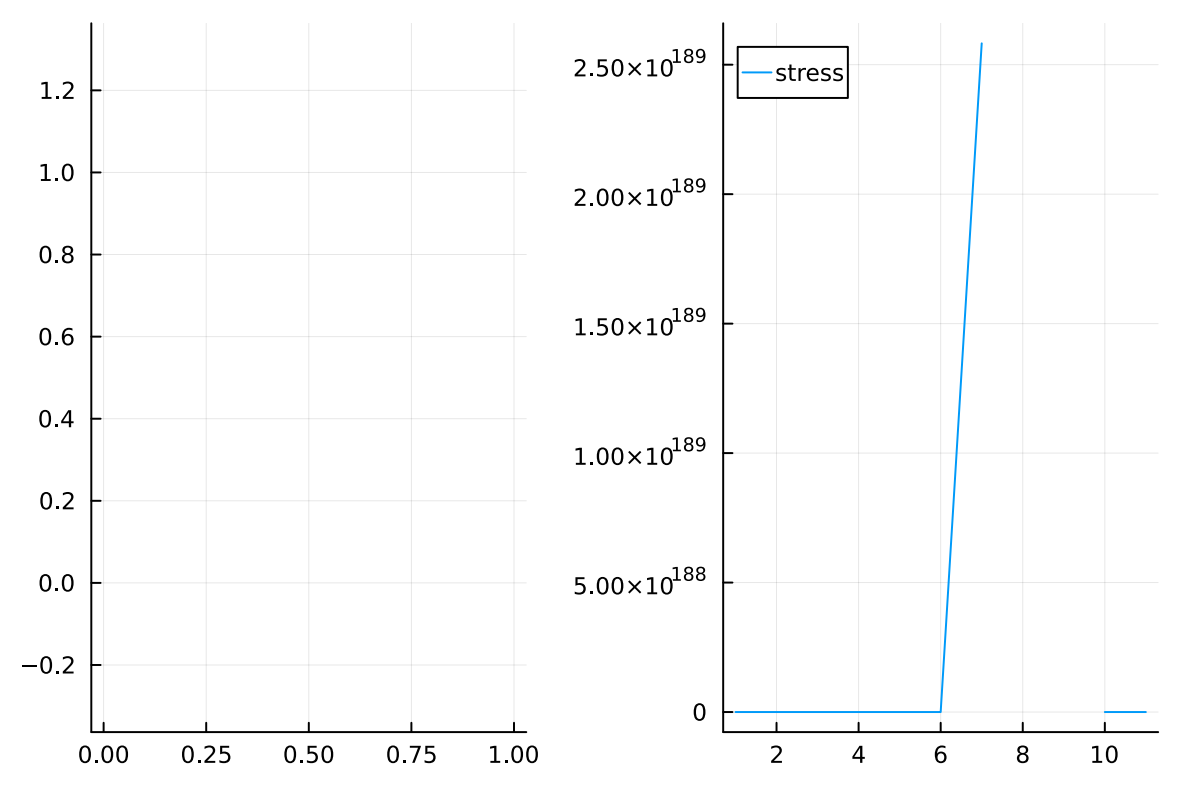

Figure 5: Wheel Graph 11, Simple Gradient Descent, $\alpha = 1.0$, Took 11 Iterations

Finally, at $\alpha = 1.0$, the algorithm does not converge at all, and the stress values explode. What's interesting is that it almost always takes 11 steps to diverge. Investigating what is going on reveals

that most of the parameters turn to *NaN* at step 10, not changing at step 11 and therefore "converging" as per our definition. The reason for this being essentially repeatable even though the initial positions are random is because the initial positions are bound to be in a range between 0 and 1.

From this, we can learn that we should always be careful when choosing the learning rate. For a simple graph, there are not as many local minima, so bigger learning rates still work, but we should not do this for a more complex graph.

Continuing with the simple gradient descent method, erll stress, $\alpha = 0.001$ and the wheel graph 11, we see that convergence is painfully slow. On the graph, we can see the vertices spreading out way more than with fr stress. This reveals a fundamental difference: ERLL stress allows vertices to be far away more than FR stress, which makes the learning rate proportionally smaller. We increase

the learning rate to $\alpha = 0.1$ and see that convergence is much faster.

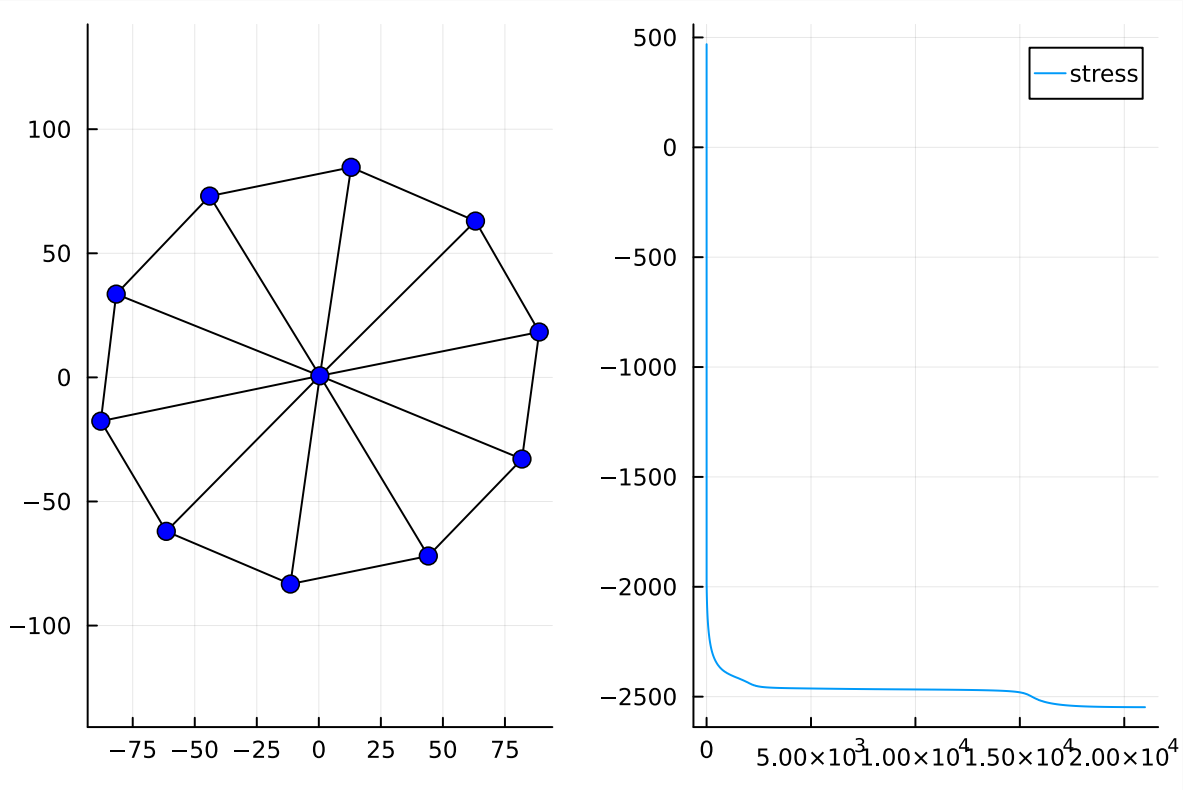

Figure 6: Wheel Graph 11, Simple Gradient Descent, $\alpha = 0.1$, Took 20994 Iterations

Considering that at $\alpha = 0.001$ compute times are reasonable for FR Stress, and at $\alpha = 0.1$ compute times are reasonable for ERLL Stress, we can say that these are good learning rates for the respective stress functions.

For simplicity reasons, we will use the same α for the nesterov gradient descent, one could of course tune this separately.

In the following, we will show some examples of graphs drawn with the nesterov gradient descent and the different stress functions. They offer a comparison between the development of the stress functions. We will use $\alpha = 0.001$ for the fr stress and $\alpha = 0.1$ for the erll stress, as we have established these are good learning rates. As a momentum decay rate, we will use $\beta = 0.9$.

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

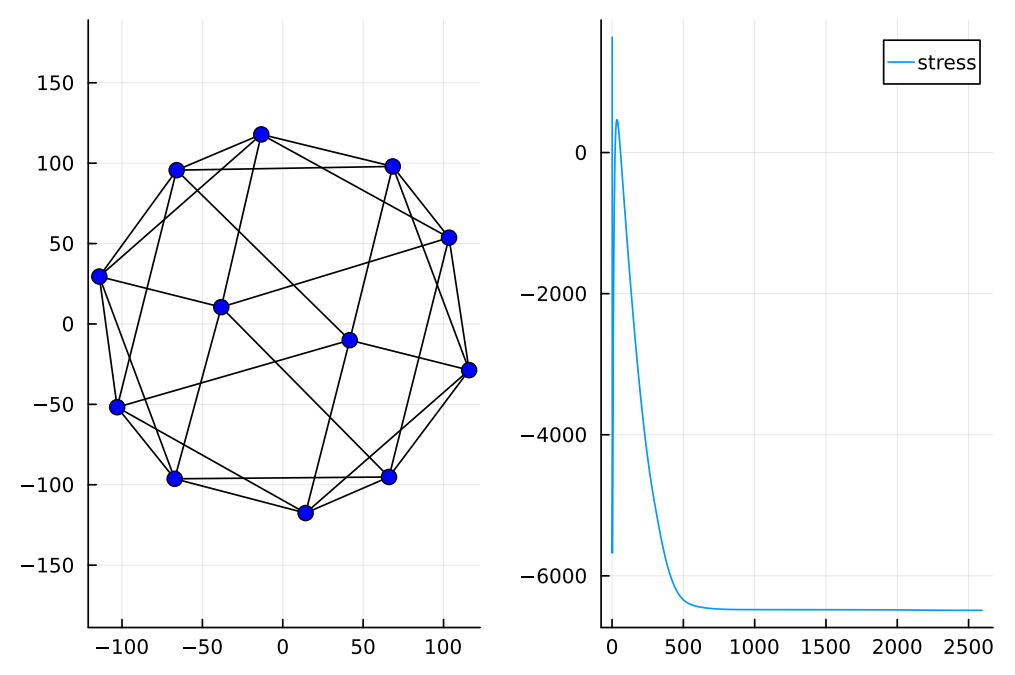

Figure 7: Icosahedral Graph, Took 2593 Iterations

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

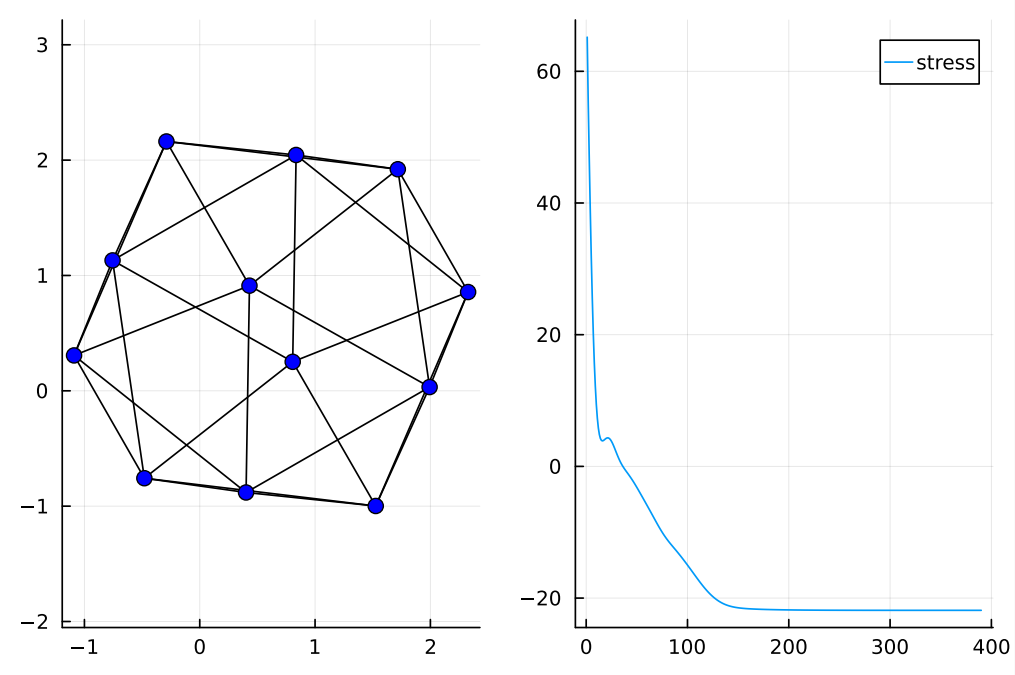

Figure 8: Icosahedral Graph, Took 389 Iterations

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

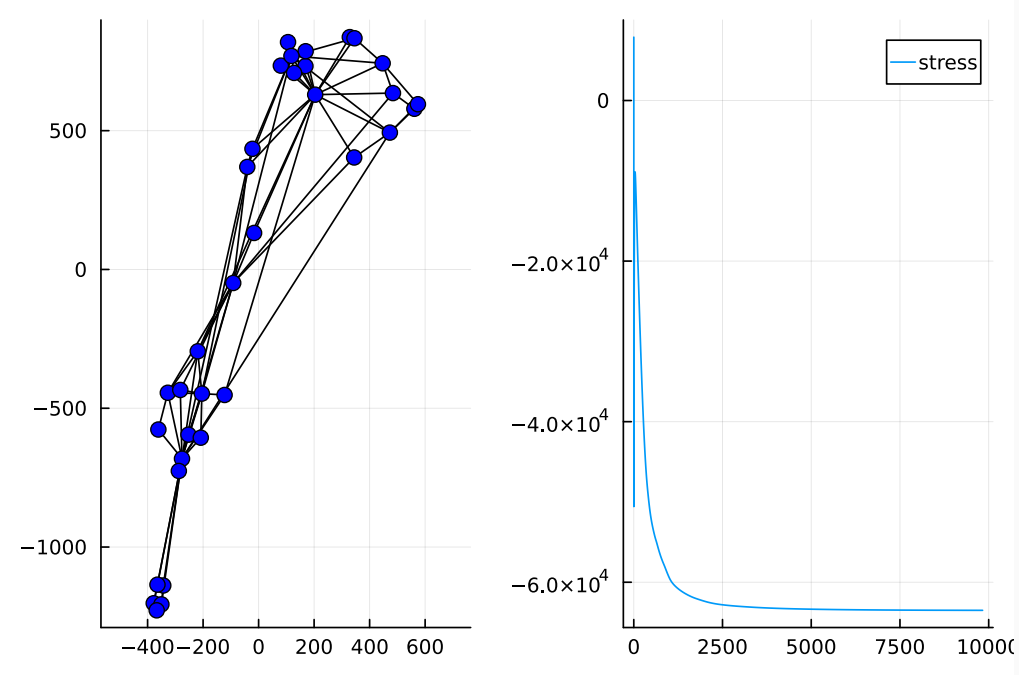

Figure 9: Karate Network, Took 9828 Iterations

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

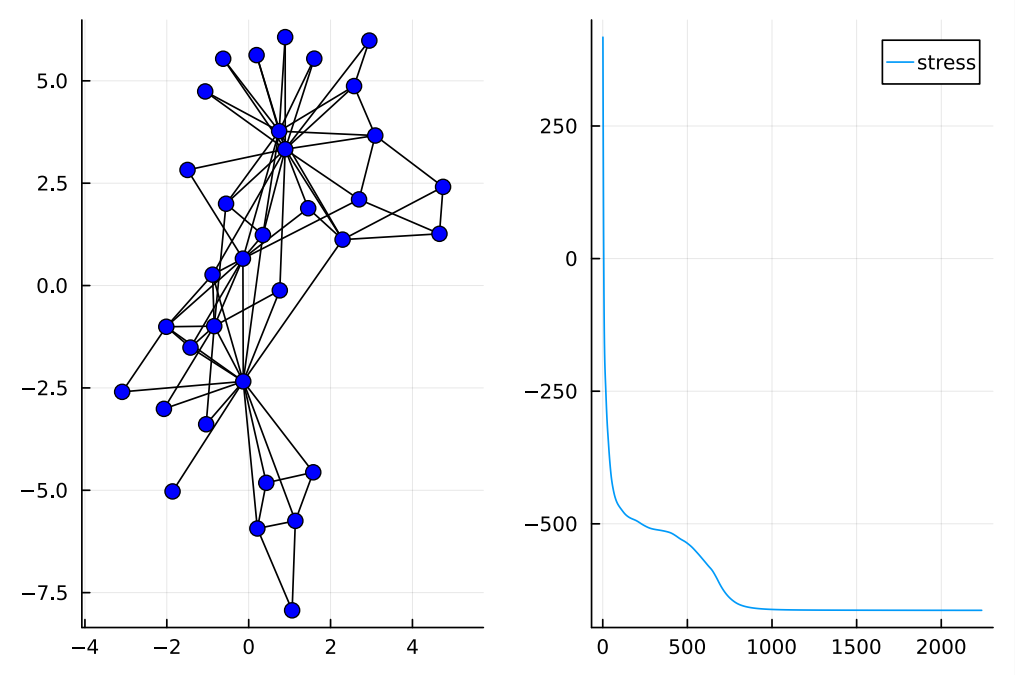

Figure 10: Karate Network, Took 2238 Iterations

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

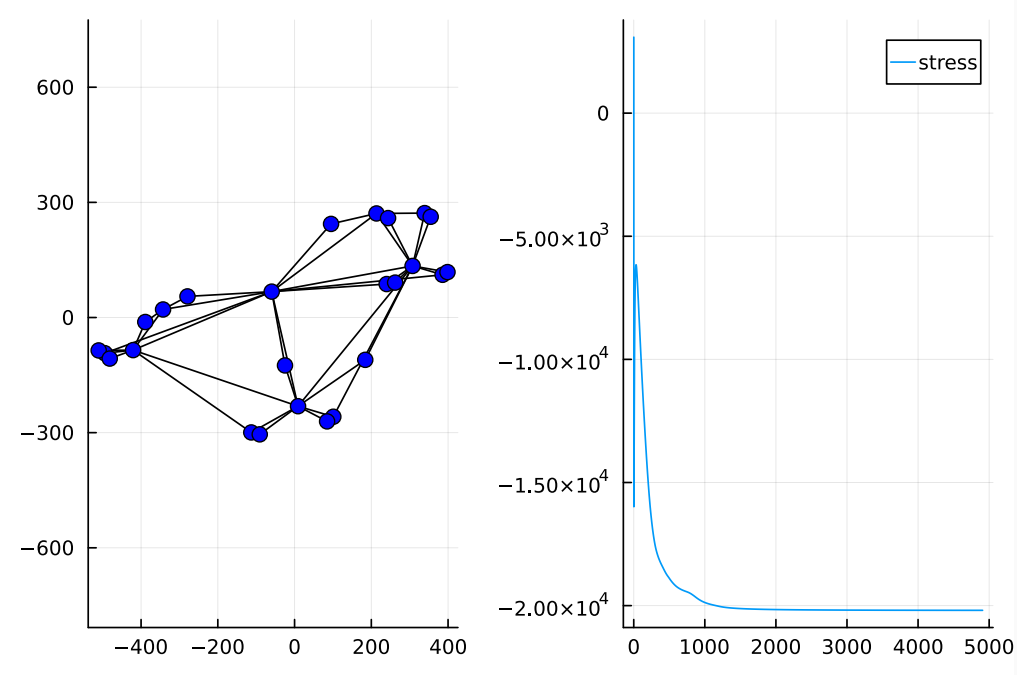

Figure 11: Dorogovtsev Mendes Graph, Took 4907 Iterations

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

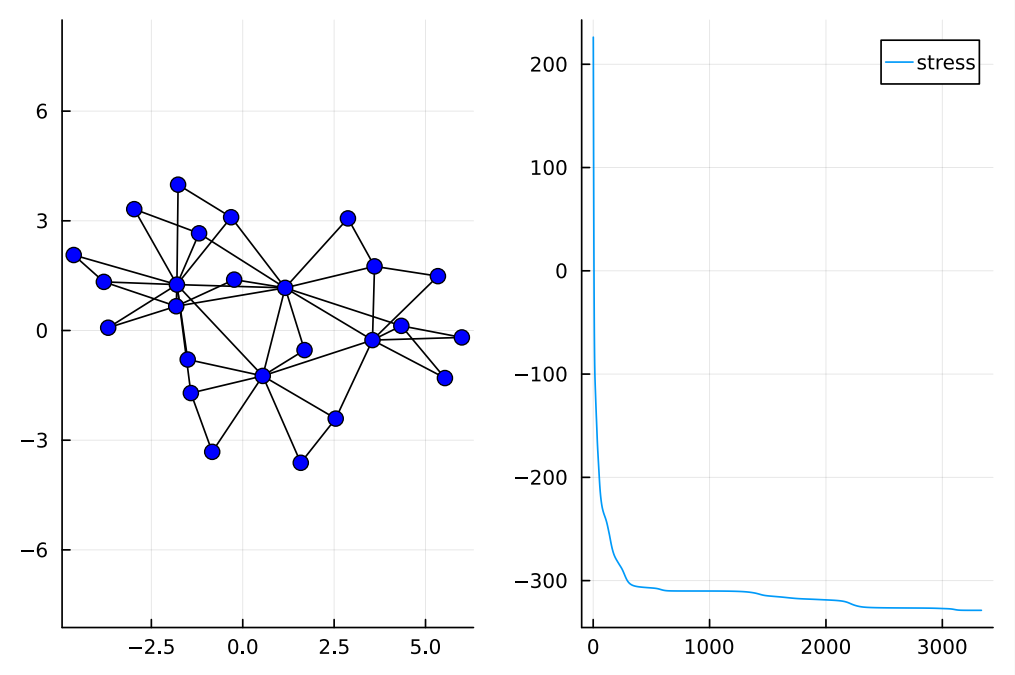

Figure 12: Dorogovtsev Mendes Graph, Took 3335 Iterations

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

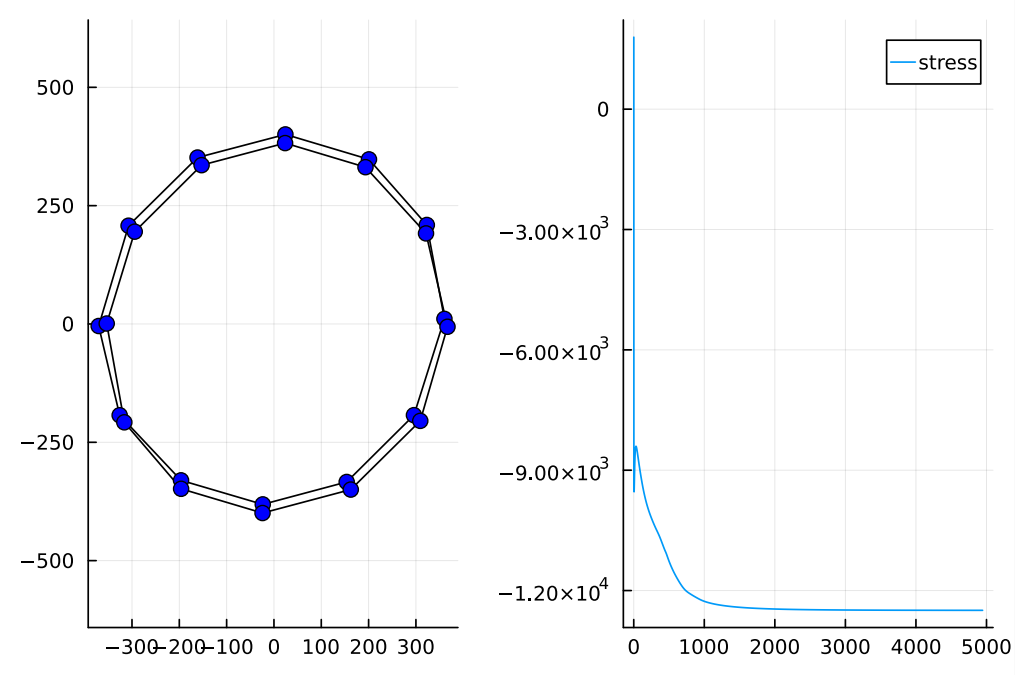

Figure 13: Circular Ladder Graph, Took 4944 Iterations

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

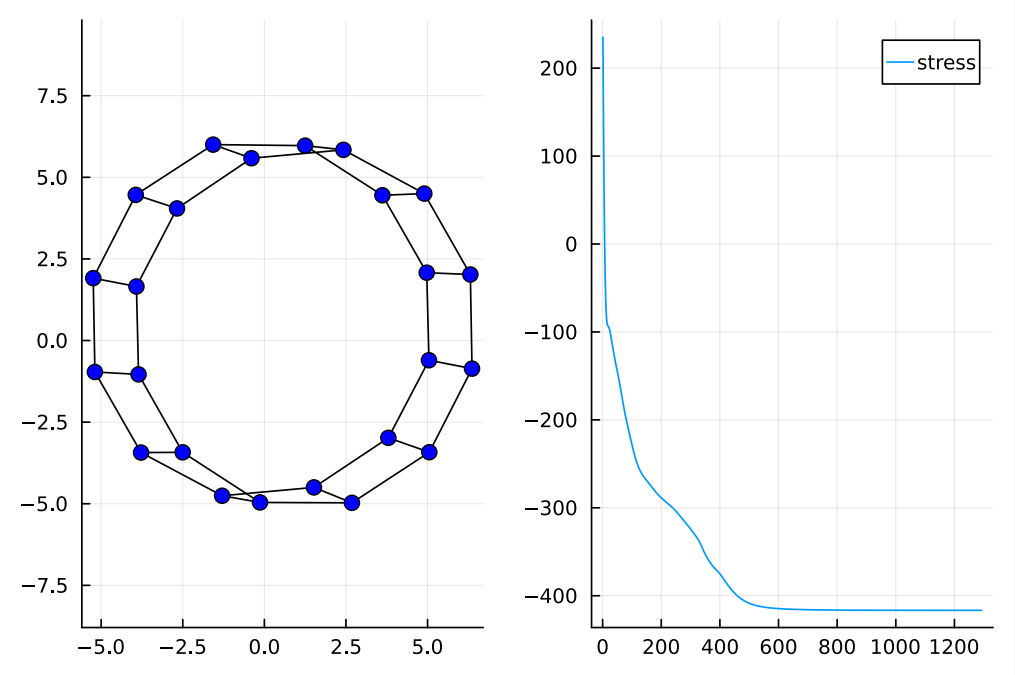

Figure 14: Circular Ladder Graph, Took 1292 Iterations

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

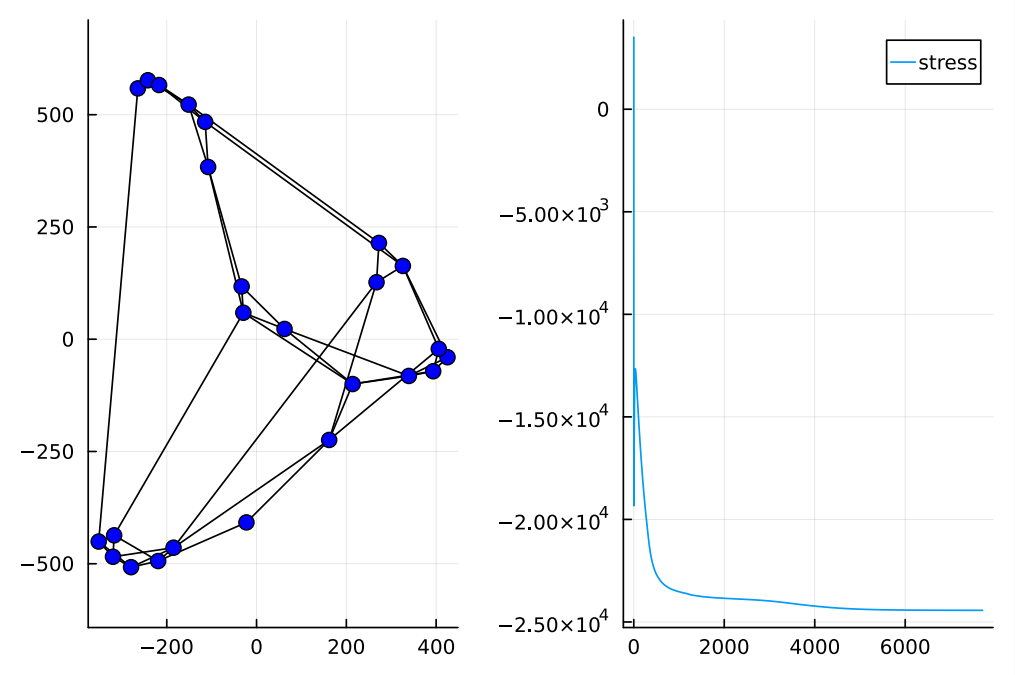

Figure 15: Watts Strogatz Graph, Took 7710 Iterations

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

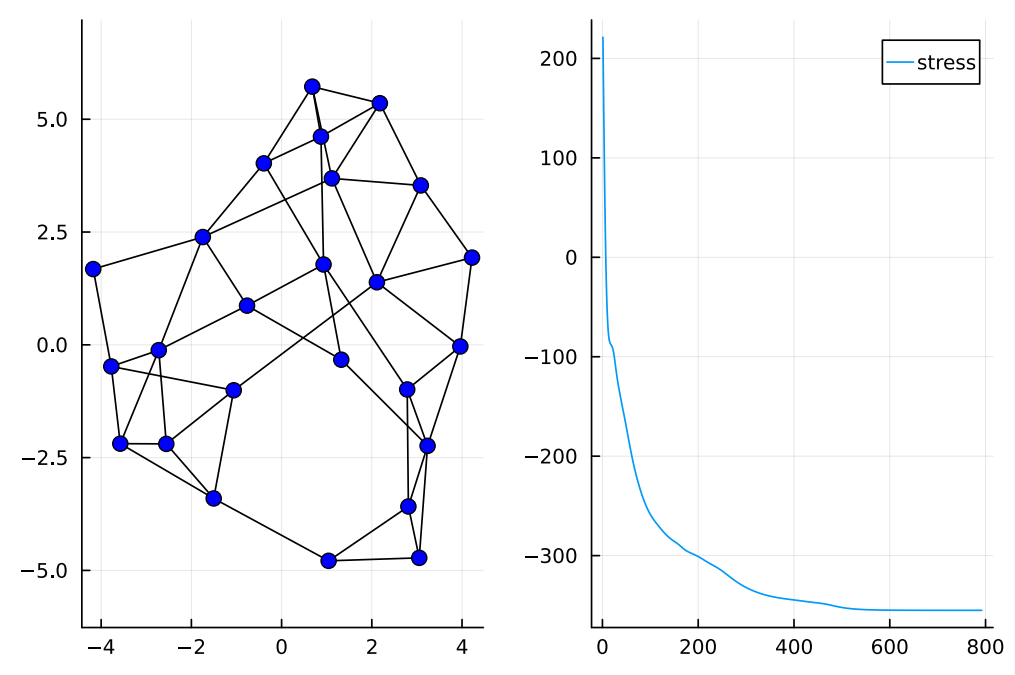

Figure 16: Watts Strogatz Graph, Took 791 Iterations

Nesterov Gradient Descent, ERLL, $\alpha = 0.1$, $\epsilon = 1e - 7$

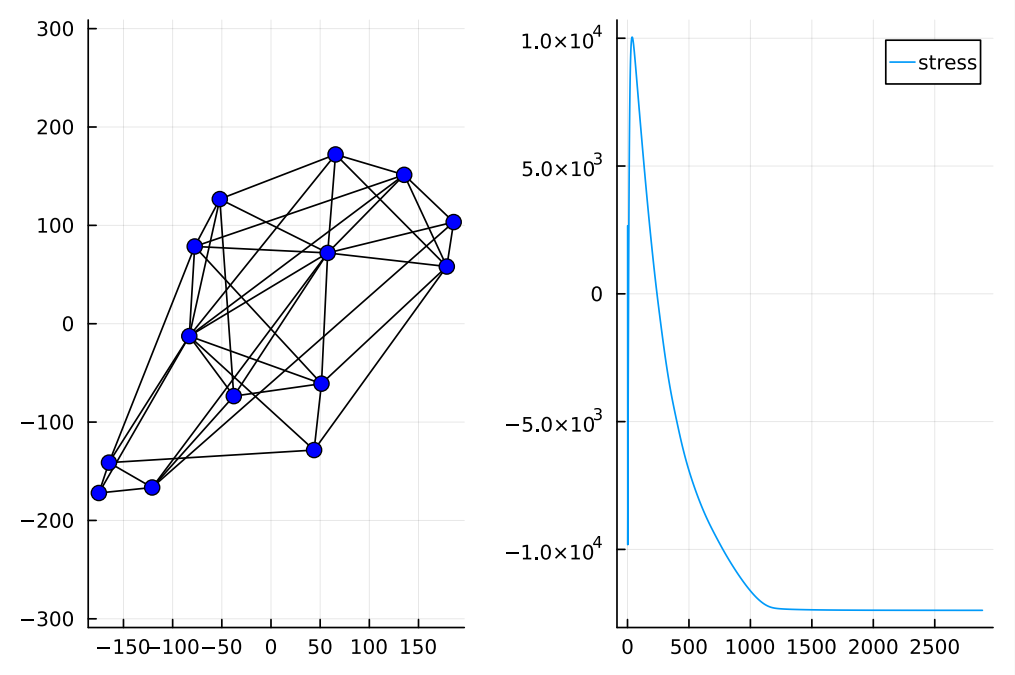

Figure 17: Static Scale Free Graph, Took 2887 Iterations

Nesterov Gradient Descent, FR, $\alpha = 0.001$, $\epsilon = 1e - 7$

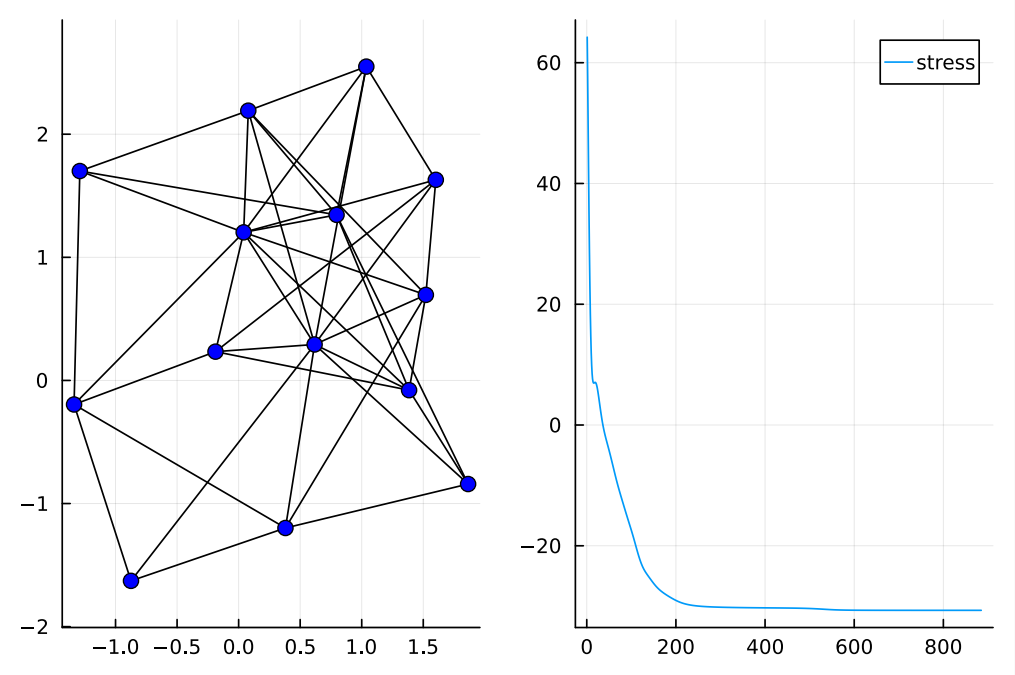

Figure 18: Static Scale Free Graph, Took 884 Iterations

Conclusion

In this project, we have explained the theory behind graph drawing, implemented the graph drawing algorithm in Julia and compared different approaches and settings.

In our implementation, FR stress is not only much faster, but also produces better results. ERLL stress just pushes vertices too far apart.

With the settings above, we have created some nice drawings of graphs. They offer true insight into the structure even with no knowledge of the graphs themselves.

However, some graphs like the circular ladder graph almost always get stuck in suboptimal positions, because they are planar in theory but in practice, the algorithm cannot untwist the two circles.

Graphs like bipartite graphs also get drawn in a very unsual way with this method.

While fun, for a truly good graph drawing algorithm, many improvements are possible.

References

[1] Andreas Noack. **"Energy Models for Drawing Clustered Small-World Graphs".** In: *Proceedings of International Symposium on Graph Drawing* (Aug. 2003).

[2] Dahua Lin and contributors. ***Distances.jl: A Julia Package for Evaluating Distances (Metrics) Between Vectors*.** Julia package, JuliaStats organization. 2014. URL: `https://github.com/JuliaStats/Distances.jl`.

[3] Michael Innes. **"Don't Unroll Adjoint: Differentiating SSA-Form Programs".** In: *CoRR* abs/1810.07951 (2018). arXiv: `1810.07951`. URL: `http://arxiv.org/abs/1810.07951`.

[4] Mirco Schönfeld and Jürgen Pfeffer. **"Fruchterman/Reingold (1991): Graph Drawing by Force-Directed Placement".** In: *Schlüsselwerke der Netzwerkforschung*. Ed. by Boris Holzer and Christian Stegbauer. Wiesbaden: Springer Fachmedien Wiesbaden, 2019, pp. 217–220. ISBN: 978-3-658-21742-6. DOI:

10.1007/978-3-658-21742-6_49. URL: https://doi.org/10.1007/978-3-658-21742-6_49.

[5] James Fairbanks et al. ***JuliaGraphs/Graphs.jl: an optimized graphs package for the Julia programming language***. 2021. URL: https://github.com/JuliaGraphs/Graphs.jl/.

[6] Simon Christ et al. **"Plots.jl – A User Extendable Plotting API for the Julia Programming Language".** In: *Journal of Open Research Software* (2023). DOI: 10.5334/jors.431. URL: https://openresearchsoftware.metajnl.com/articles/10.5334/jors.431.